

Arduino Uno Pinout Configuration

Digital I/O (0–13):

- Can be set as INPUT or OUTPUT using `pinMode()`.
- Pins marked with ~ (3, 5, 6, 9, 10, 11) support PWM (Pulse Width Modulation) — useful for dimming LEDs or controlling motor speed via `analogWrite()`.
- Pin 0 (RX) and Pin 1 (TX) are used for serial communication (UART) — avoid using them while uploading code or using `Serial.print()`.
- Pin 13 connects to the built-in LED on the board — no external LED needed to test blink code.

Analog Input (A0–A5):

- Read varying voltages between 0–5V and convert them to a digital value from 0–1023 using the onboard 10-bit ADC (`analogRead()`).
- Commonly used for potentiometers, LDRs, temperature sensors, etc.
- A4 (SDA) and A5 (SCL) double as the I2C communication pins for connecting sensors/modules like OLED displays or RTC modules.

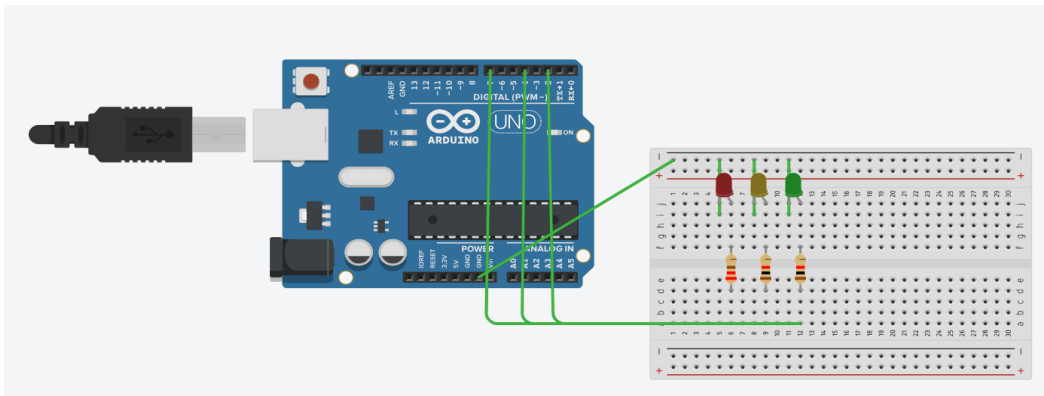
Power Header:

- **5V / 3.3V** — regulated output to power external components.
- **GND** — multiple ground pins for completing circuits.
- **Vin** — input pin to power the board externally (7–12V), bypassing the USB/DC jack.
- **RESET** — pulling this LOW resets the microcontroller.

Other key parts:

- **USB port** — connects to a computer for uploading sketches and serial communication (handled by a secondary ATmega16U2 chip).
- **DC power jack** — accepts 7–12V input for standalone operation.
- **ICSP header** — used for direct programming of the microcontroller (In-Circuit Serial Programming), bypassing the bootloader.
- **Reset button** — manually restarts the running program.

CIRCUIT CONNECTIONS



ALGORITHM:

1. Start
2. Connect the three LEDs (Red, Yellow, Green) to Arduino through resistors.
3. Set the LED pins as OUTPUT.
4. Turn ON the Red LED and keep Yellow and Green OFF.
5. Wait for a few seconds.
6. Turn ON the Yellow LED and turn Red OFF.
7. Wait for a few seconds.
8. Turn ON the Green LED and turn Yellow OFF.
9. Wait for a few seconds.
10. Repeat the sequence continuously.
11. Stop.

PROCEDURE:

1. Open the Tinkercad Circuits web application and create a new circuit.
2. Drag and drop an Arduino Uno, a breadboard, an LED and a resistor onto the workspace.
3. Place the LED on the breadboard with its cathode (shorter leg / flat edge) in one row and anode (longer leg) in another row.
4. Connect one leg of the 220 Ω resistor to the LED's anode row, and the other end of the resistor to digital pin 13 of the Arduino using a jumper wire.
5. Connect the LED's cathode row to the ground rail of the breadboard, and connect the ground rail to the GND pin of the Arduino using a jumper wire.
6. Verify all connections carefully as shown in the circuit diagram (Fig. 1).
7. Click on the "Code" editor in Tinkercad and select the "Text" / Arduino IDE mode.
8. Type the Arduino sketch given in Section 6 into the code editor.
9. Click the "Start Simulation" button to run the program and observe the LED blinking behaviour.
10. Modify the program (delay values / pin patterns) to design different blinking patterns and re-simulate.
11. Note down the observations and save/screenshot the simulation result.

ARDUINO PROGRAM

// Arduino Program for 3 LEDs on Pins 2, 4, and 7

```
int led1 = 2;
```

```
int led2 = 4;
```

```
int led3 = 7;
```

```
void setup() {  
  pinMode(led1, OUTPUT);  
  pinMode(led2, OUTPUT);  
  pinMode(led3, OUTPUT);  
}
```

```
void loop() {  
  // LED 1 ON
```

```
digitalWrite(led1, HIGH);
delay(500);
digitalWrite(led1, LOW);
```

```
// LED 2 ON
digitalWrite(led2, HIGH);
delay(500);
digitalWrite(led2, LOW);
```

```
// LED 3 ON
digitalWrite(led3, HIGH);
delay(500);
digitalWrite(led3, LOW);
```

```
}
```

OBSERVATION TABLE:

Sr. No.	Pin State (digitalWrite)	Delay (ms)	Observed LED Status
1	Pin 13 = HIGH	1000	LED glows (ON)
2	Pin 13 = LOW	1000	LED turns off (OFF)
3	Repeats continuously	—	LED blinks ON-OFF every 1 second

RESULT:

The LED connected to pin 13 of the Arduino Uno was successfully interfaced and programmed using Tinkercad. The LED blinked ON and OFF at an interval of one second as expected, and different blinking patterns were successfully designed by modifying the delay and digitalWrite statements in the program.

PRECAUTIONS:

- Always connect the LED with correct polarity — anode to the resistor/pin side, cathode to ground.
- Never connect an LED directly to a digital pin without a current-limiting resistor.
- Ensure pinMode() is correctly set to OUTPUT before using digitalWrite().
- Verify all breadboard connections before running the simulation to avoid short circuits.
- Save the Tinkercad circuit periodically to avoid loss of work.

CONCLUSION:

Thus, the LED was successfully interfaced with Arduino Uno using Tinkercad, and the fundamental concept of digital output control (pinMode, digitalWrite, delay) was studied and verified practically. Different LED blinking patterns were also designed and simulated, forming the foundation for more advanced embedded systems experiments.

(B)AIM:

To design the circuit and write a program (WAP) to interface switches and push buttons using pull-up and pull-down resistors with Arduino Uno.

OBJECTIVE:

To design and implement a circuit using Arduino Uno to interface switches and push buttons with pull-up and pull-down resistors, and to write a program to detect and control their input states.

HARDWARE & SOFTWARE USED:

Sr. No.	Component	Specification / Quantity
1	Arduino Uno	ATmega328P based development board
2	Breadboard	Half / Full size
3	LED	5 mm, any colour (status indicator)
4	Switch / Push Button	Momentary tactile push button
5	Resistor	10 K Ω (one each for pull-up & pull-down) and 470 Ω (for LED)
6	Jumper Wires	Male-to-Male, as required
7	Software	Tinkercad Circuits (web-based simulator) / Arduino IDE

THEORY:

A switch or push button, by itself, produces an undefined ('floating') logic level when it is open, because the digital input pin is left disconnected from both 5V and GND. Stray electrical noise can then cause the pin to randomly read HIGH or LOW, giving unreliable results. To avoid this, a resistor is used to hold the pin at a known, defined voltage level when the switch is not pressed. This resistor can be connected in one of two ways - as a pull-up or a pull-down resistor.

(a) Pull-Down Resistor Configuration

In this configuration, a resistor (10 K Ω) is connected between the input pin and GND, while the switch connects the same pin to 5V. When the switch is open, the resistor pulls the pin voltage down to 0V (LOW). When the switch is pressed, the pin is directly connected to 5V, so it reads HIGH.

- Switch open $\rightarrow$ Input pin = LOW (0)
- Switch pressed $\rightarrow$ Input pin = HIGH (1)

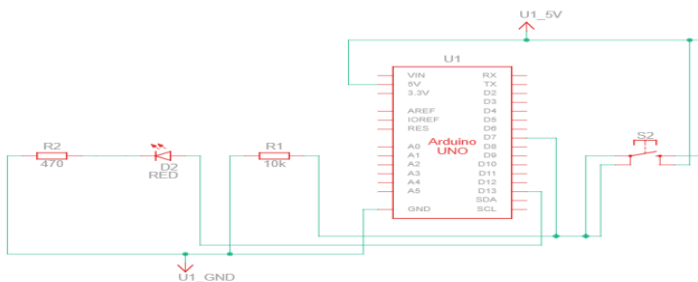
(b) Pull-Up Resistor Configuration

Here, the resistor (10 K Ω) is connected between the input pin and 5V, while the switch connects the same pin to GND. When the switch is open, the resistor pulls the pin voltage up to 5V (HIGH). When the switch is pressed, the pin is directly connected to GND, so it reads LOW.

- Switch open $\rightarrow$ Input pin = HIGH (1)
- Switch pressed $\rightarrow$ Input pin = LOW (0)

Arduino Uno also provides a built-in (internal) pull-up resistor of approximately 20–50 k Ω on every digital pin, which can be enabled in software using `pinMode(pin, INPUT_PULLUP)` - this eliminates the need for an external pull-up resistor, although an external pull-down resistor has no internal equivalent and must always be wired manually.

CIRCUIT DIAGRAM - PULL-DOWN CONFIGURATION:



PROCEDURE:

1. Open the Tinkercad Circuits web application and create a new circuit.
2. Drag and drop an Arduino Uno, a breadboard, a push button, an LED and two 10 K Ω resistors onto the workspace.
3. For the pull-down circuit: connect one terminal of the switch to the 5V rail, and the other terminal to digital pin 7 as well as to one end of a 10 K Ω resistor, whose other end goes to GND.
4. For the pull-up circuit: connect one terminal of the switch to GND, and the other terminal to digital pin 8 as well as to one end of a 10 K Ω resistor, whose other end goes to the 5V rail.
5. Connect the LED anode to digital pin 13 (through the breadboard) and the cathode to GND, to serve as a visual status indicator.
6. Verify all connections carefully as shown in Fig. 1 and Fig. 2.
7. Open the Code editor in Tinkercad and type the Arduino sketch (Section 7) for the desired configuration.
8. Click Start Simulation and test the circuit by clicking the push button.
9. Observe the LED status for both the pressed and released states of the switch.
10. Repeat the simulation for the alternate configuration (pull-up / pull-down) and note the difference in logic levels.

CODE:

(a) Using external pull-down resistor (Pin 7):

// Program : Push Button Interfacing using PULL-DOWN Resistor

// Board : Arduino Uno

// Purpose : Turn ON the LED when the push button is pressed


```
// and turn it OFF when the button is released.
// Logic :
// Switch Open -> Pin 7 = LOW -> LED OFF
// Switch Pressed -> Pin 7 = HIGH -> LED ON
// Declare a constant variable for the push button pin.
// The push button is connected to Digital Pin 7.
const int buttonPin = 7;
// Declare a constant variable for the LED pin.
// The built-in LED (or external LED) is connected to Digital Pin 13.
const int ledPin = 13;
void setup() {
  // The setup() function runs only once
  // when the Arduino is powered ON or reset.
  // Configure the push button pin as an INPUT.
  // An external 1kΩ pull-down resistor keeps the pin LOW
  // when the button is not pressed.
  pinMode(buttonPin, INPUT);
  // Configure the LED pin as an OUTPUT
  // so the Arduino can control the LED.
  pinMode(ledPin, OUTPUT);
}
void loop() {
  // The loop() function runs continuously.
  // Read the current state of the push button.
  // digitalRead() returns:
  // HIGH (5V) -> Button Pressed
  // LOW (0V) -> Button Released
  int buttonState = digitalRead(buttonPin);
  // Check whether the button is pressed.
  if (buttonState == HIGH) {
    // If the button is pressed,
    // turn the LED ON.
    digitalWrite(ledPin, HIGH);
  } else {
    // If the button is not pressed,
    // turn the LED OFF.
    digitalWrite(ledPin, LOW);
  }
  // The loop repeats continuously,
  // checking the button status again and again.
}
```

(b) Using external pull-up resistor (Pin 8):

```
// Program : Push Button Interfacing using PULL-UP Resistor
// Board : Arduino Uno
// Working Logic:
// Switch Open -> Pin 8 = HIGH -> LED ON
```

```
// Switch Pressed -> Pin 8 = LOW -> LED OFF
// Note:
// An external 1kΩ pull-up resistor is connected between
// Arduino Pin 8 and +5V.
// The push button connects Pin 8 to GND when pressed.
// Store the push button pin number (Digital Pin 8)
const int buttonPin = 8;
// Store the LED pin number (Built-in LED on Digital Pin 13)
const int ledPin = 13;
void setup()
{
  // Configure button pin as INPUT.
  // Since an external pull-up resistor is used,
  // INPUT mode is selected instead of INPUT_PULLUP.
  pinMode(buttonPin, INPUT);
  // Configure LED pin as OUTPUT
  // so Arduino can control the LED.
  pinMode(ledPin, OUTPUT);
}
void loop()
{
  // Read the current state of the push button.
  // The value will be:
  // HIGH -> Button released
  // LOW -> Button pressed
  int buttonState = digitalRead(buttonPin);
  // Check whether the button is pressed.
  if (buttonState == LOW)
  {
    // Button is pressed.
    // Pin 8 is connected to GND.
    // Turn OFF the LED by sending LOW to Pin 13.
    digitalWrite(ledPin, LOW);
  }
  else
  {
    // Button is released.
    // Pull-up resistor keeps Pin 8 at HIGH (+5V).
    // Turn ON the LED by sending HIGH to Pin 13.
    digitalWrite(ledPin, HIGH);
  }
  // The loop repeats continuously,
  // allowing Arduino to monitor the button at all times.
}
```

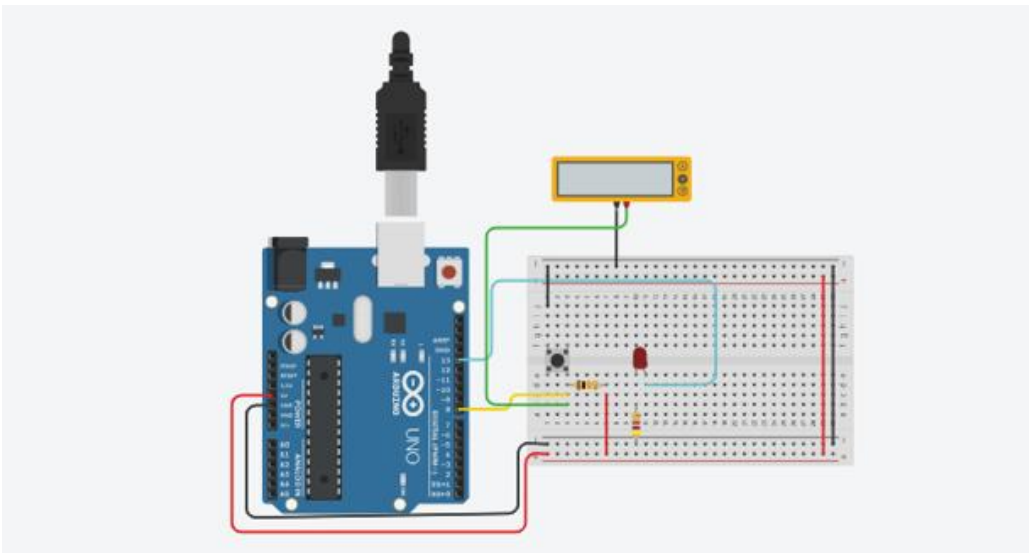

(c) Using Arduino's internal pull-up resistor (alternate method, no external resistor):

```
// Alternate method: using Arduino's INTERNAL pull-up resistor
// No external resistor required for this variation
const int buttonPin = 8;
const int ledPin    = 13;

void setup() {
  pinMode(buttonPin, INPUT_PULLUP); // Enables internal ~20-50k pull-up
  pinMode(ledPin, OUTPUT);
}

void loop() {
  if (digitalRead(buttonPin) == LOW) { // Pressed = LOW
    digitalWrite(ledPin, HIGH);
  } else {
    digitalWrite(ledPin, LOW);
  }
}
```

OUTPUT



OBSERVATION TABLE:

Configuration	Switch State	Pin Logic Level	LED Status
Pull-down (Pin 7)	Open (not pressed)	LOW (0)	OFF
Pull-down (Pin 7)	Pressed	HIGH (1)	ON
Pull-up (Pin 8)	Open (not pressed)	HIGH (1)	ON
Pull-up (Pin 8)	Pressed	LOW (0)	OFF

RESULT:

The push button was successfully interfaced with Arduino Uno using both pull-up and pull-down resistor configurations. The digital pin reliably read a defined HIGH or LOW logic level in both the pressed and released states, and the LED connected to pin 13 correctly reflected the button status in each case, confirming proper circuit design and program execution.

PRECAUTIONS:

1. Never leave a digital input pin floating (unconnected) — always use a pull-up or pull-down resistor.
2. Ensure the resistor value (10 K Ω) is correctly placed in series to limit current and define logic levels.
3. Check switch orientation on the breadboard — the two pairs of legs are internally connected in specific patterns.
4. Match the polarity of the LED (anode/cathode) correctly to avoid incorrect readings.
5. Verify pinMode() settings (INPUT vs INPUT_PULLUP) match the physical circuit configuration used.

CONCLUSION:

Thus, the circuit for interfacing a switch/push button using pull-up and pull-down resistor configurations was designed and successfully tested with Arduino Uno. Both methods were found to reliably eliminate the floating input problem, and the program (WAP) accurately detected the button state and controlled the LED output accordingly, forming a fundamental building block for digital input handling in embedded systems.